

設計ドキュメント

1. サービス概要

目的・課題解決

- RSS フィードから最新の技術情報を取得し、自動的に要約・分類する。
- 生成 AI を活用して、英語・日本語の技術情報をタグ付けし、ユーザーが手軽に情報を得られるようにする。
- 要約した情報をユーザーに通知する。
- ユーザーが手動で記事を送信し、要約をリクエストできる機能を提供する。

想定ユーザー

- エンジニア
- 初期段階では開発者自身が利用

主要機能

- RSS から最新の技術情報を取得
- 生成 AI によるタグ付けと要約
- ユーザーが手動で要約リクエストを送信可能
- ユーザーへの通知 (LINE, Slack)

2. アーキテクチャ

記事「[私のよく使うソフトウェアアーキテクチャの雛型](#)」を参考に、以下の4層構造を採用。

- **プレゼンテーション層**: ソフトウェアの入出力を担当
- **アプリケーション層**: ソフトウェアのユースケースを担当
- **ドメイン層**: ビジネスのルールや制約、プロセスを担当
- **インフラストラクチャー層**: 技術的関心ごとの全般を担当

技術スタック

- **フロントエンド**: TypeScript, React, Next.js (将来的な追加)
- **バックエンド**: Golang
- **生成 AI**: DeepSeek API
- **インフラ**: AWS CDK v2 (TypeScript)
- **データベース**: NoSQL (DynamoDB)

クラウドサービス

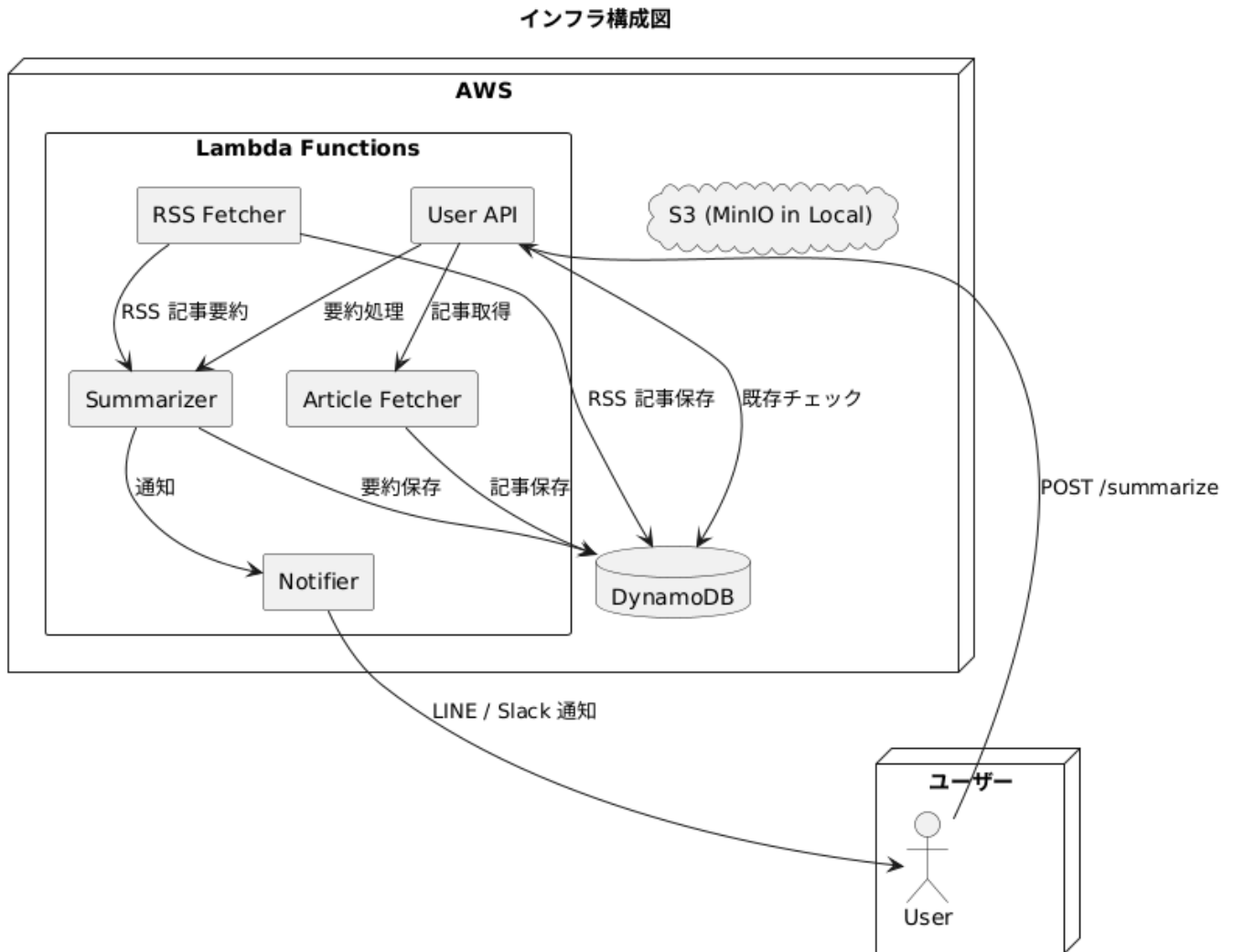
- AWS (無料枠を最大限活用)

デプロイ

- サーバーレスアーキテクチャを優先
- AWS CDK v2 を使用し、インフラをコード管理 (IaC)

- フロントエンドは Next.js の静的サイトを S3 + CloudFront でホスティング（将来的な追加）

3. インフラ構成図



► UML

```

@startuml
title インフラ構成図

node "ユーザー" {
    actor User
}

node "AWS" {
    database "DynamoDB" as DB
    cloud "S3 (MinIO in Local)" as S3
    rectangle "Lambda Functions" {
        rectangle "RSS Fetcher" as Lambda1
        rectangle "Article Fetcher" as Lambda2
        rectangle "Summarizer" as Lambda3
        rectangle "Notifier" as Lambda4
        rectangle "User API" as Lambda5
    }
}
  
```

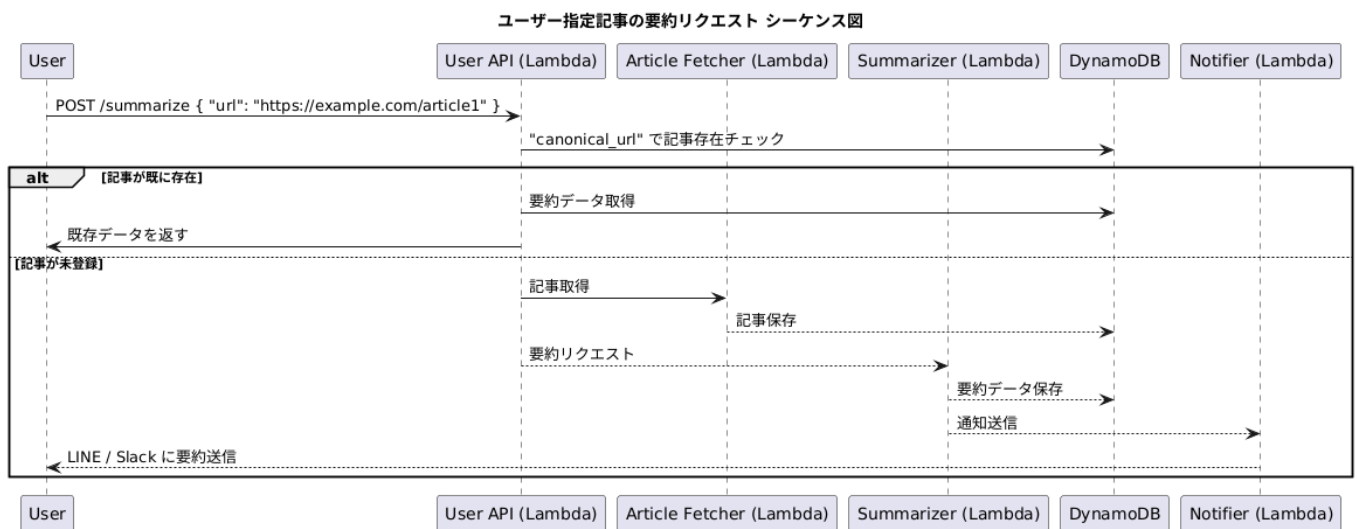
```

User --> Lambda5 : "POST /summarize"
Lambda5 --> DB : "既存チェック"
Lambda5 --> Lambda2 : "記事取得"
Lambda2 --> DB : "記事保存"
Lambda5 --> Lambda3 : "要約処理"
Lambda3 --> DB : "要約保存"
Lambda3 --> Lambda4 : "通知"
Lambda4 --> User : "LINE / Slack 通知"

Lambda1 --> DB : "RSS 記事保存"
Lambda1 --> Lambda3 : "RSS 記事要約"
@enduml

```

4. シーケンス図



► UML

```

@startuml
title ユーザー指定記事の要約リクエスト シーケンス図

participant User
participant "User API (Lambda)" as Lambda5
participant "Article Fetcher (Lambda)" as Lambda2
participant "Summarizer (Lambda)" as Lambda3
participant DynamoDB as DB
participant "Notifier (Lambda)" as Lambda4

User -> Lambda5: POST /summarize { "url": "https://example.com/article1" }
Lambda5 -> DB: "canonical_url" で記事存在チェック
alt 記事が既に存在
    Lambda5 -> DB: 要約データ取得
    DB --> Lambda5: 既存データを返す
else 記事が未登録
    Lambda5 -> Lambda2: 記事取得
    Lambda2 --> DB: 記事保存
    Lambda5 --> Lambda3: 要約リクエスト
    Lambda3 --> DB: 要約データ保存
    Lambda3 --> Lambda4: 通知送信
end
Lambda4 --> User: LINE / Slack に要約送信

```

```
Lambda3 --> DB: 要約データ保存
Lambda3 --> Lambda4: 通知送信
Lambda4 --> User: LINE / Slack に要約送信
end
@enduml
```

5. データベース設計

✅ **articles** テーブル (要約記事を保存)

Partition Key (canonical_url or original_url)	Title	Summary	Tags	Created At
https://example.com/article1	"AIの未来"	"要約結果..."	["AI", "Tech"]	2025-03-15
https://example.com/alt-article1	"AIの未来"	"要約結果..."	["AI", "Tech"]	2025-03-16

✅ **users** テーブル (ユーザー管理)

Partition Key (user_id)	Plan	Requests Limit	Used Requests	Last Reset
user123	free	5	3	2025-03-15
user456	premium	100	10	2025-03-15

✅ **tags** テーブル (タグ情報)

Partition Key (tag_name)	Created At
AI	2025-03-10
Cloud	2025-03-10

✅ **article_tags** テーブル (タグと記事の関連)

Partition Key (tag_name)	Sort Key (article_id)
AI	article_abc123
Cloud	article_abc123
Security	article_def456

6. 監視・運用

- 記事取得時のエラー (403, 404, タイムアウト) を CloudWatch Logs に記録
- 要約処理の実行時間を CloudWatch Metrics で監視
- DynamoDB のスループットを監視し、負荷がかかりすぎないように調整

7. まとめ

- ✓ Canonical URL を優先して重複チェック（ない場合は `original_url` を代用）
- ✓ 最初のリリースでは Embedding を使わず、拡張しやすいテーブル設計にする
- ✓ 将来的にレコメンドや類似検索を追加できる形で設計
- ✓ 監視・エラーハンドリングを考慮し、運用コストを抑える